

LAMPIRAN A - DOKUMENTASI KODE SUMBER (BAGIAN 2)

Komponen React.js - Web Dashboard

File: `react-dashboard/src/contexts/AuthContext.jsx`

Deskripsi: Context provider untuk autentikasi Firebase, session management (1 jam), presence system, dan Role-Based Access Control (RBAC) dengan 5 level role.

```
import React, { createContext, useContext, useState, useEffect, useCallback } from "react";
import { onAuthStateChanged, signInWithEmailAndPassword, signOut } from "firebase/auth";
import { ref, onValue, set, get, remove, onDisconnect } from "firebase/database";
import { auth, database } from "../firebase";

const AuthContext = createContext(null);
const SESSION_DURATION = 60 * 60 * 1000; // 1 jam
const LOGIN_TIMESTAMP_KEY = "solardryer_login_timestamp";

const ROLE_CONFIG = {
  guess: { level: 0, tabs: ["dashboard"] },
  view_data: { level: 1, tabs: ["dashboard", "data"] },
  eksekusi_data: { level: 2, tabs: ["dashboard", "control", "data"] },
  report_data: { level: 3, tabs: ["dashboard", "data"] },
  admin: { level: 4, tabs: ["dashboard", "control", "data", "management"] },
};

export function canAccessTab(role, tabId) {
  const config = ROLE_CONFIG[role];
  if (!config) return false;
```

```

    return config.tabs.includes(tabId);
  }

  export function canExport(role) {
    return role === "report_data" || role === "admin";
  }

  export function canManage(role) {
    return role === "admin";
  }

  export function AuthProvider({ children }) {
    const [currentUser, setCurrentUser] = useState(null);
    const [userRole, setUserRole] = useState(null);
    const [loading, setLoading] = useState(true);
    const [sessionTimer, setSessionTimer] = useState(null);

    const logout = useCallback(async () => {
      try {
        if (sessionTimer) clearTimeout(sessionTimer);
        localStorage.removeItem(LOGIN_TIMESTAMP_KEY);
        if (auth.currentUser) {
          const presRef = ref(database, `presence/${auth.currentUser.uid}`);
          await remove(presRef).catch(() => {});
        }
        setUserRole(null);
        setCurrentUser(null);
        await signOut(auth);
      } catch (err) { console.error("Logout error:", err); }
    }, [sessionTimer]);

    const startSessionTimer = useCallback((loginTimestamp) => {
      if (sessionTimer) clearTimeout(sessionTimer);
      const elapsed = Date.now() - loginTimestamp;
      const remaining = SESSION_DURATION - elapsed;
      if (remaining <= 0) { logout(); return; }
      const timer = setTimeout(() => { logout(); }, remaining);
      setSessionTimer(timer);
    }, [sessionTimer, logout]);

```

```

const login = async (email, password) => {
  const userCredential = await signInWithEmailAndPassword(auth, email, password);
  const user = userCredential.user;
  const now = Date.now();
  localStorage.setItem(LOGIN_TIMESTAMP_KEY, now.toString());
  try {
    await set(ref(database, `users/${user.uid}`), {
      email: user.email,
      displayName: user.displayName || user.email.split("@")[0],
      lastLogin: now,
    });
    const aksesSnap = await get(ref(database, `user_akses/${user.uid}`));
    if (!aksesSnap.exists()) {
      await set(ref(database, `user_akses/${user.uid}`), { role: "guess", updatedAt: now });
    }
  } catch (err) { console.warn("Could not write user data:", err); }
  return user;
};

useEffect(() => {
  const unsubAuth = onAuthStateChanged(auth, (user) => {
    if (user) {
      setCurrentUser(user);
      const storedTimestamp = localStorage.getItem(LOGIN_TIMESTAMP_KEY);
      if (storedTimestamp) {
        const ts = parseInt(storedTimestamp, 10);
        if (Date.now() - ts >= SESSION_DURATION) { logout(); return; }
        startSessionTimer(ts);
      } else {
        const now = Date.now();
        localStorage.setItem(LOGIN_TIMESTAMP_KEY, now.toString());
        startSessionTimer(now);
      }
      const presRef = ref(database, `presence/${user.uid}`);
      set(presRef, { email: user.email, displayName: user.displayName || user.email.split("@")[0], lastSeen: Date.now() });
      onDisconnect(presRef).remove();
      const roleRef = ref(database, `user_akses/${user.uid}/role`);
      onValue(roleRef, (snapshot) => {

```

```

        setUserRole(snapshot.exists() ? snapshot.val() : "guess");
        setLoading(false);
    });
    } else {
        setCurrentUser(null); setUserRole(null); setLoading(false);
    }
    });
    return () => unsubAuth();
}, []);

return (
    <AuthContext.Provider value={{ currentUser, userRole, loading, login, logout }}>
        {children}
    </AuthContext.Provider>
);
}

export function useAuth() {
    const context = useContext(AuthContext);
    if (!context) throw new Error("useAuth must be used within an AuthProvider");
    return context;
}

```

File: `react-dashboard/src/components/SensorCard.jsx`

Deskripsi: Komponen kartu sensor untuk menampilkan rata-rata suhu/kelembaban dan data individual per sensor.

```

import React from "react";
import { WiThermometer, WiHumidity } from "react-icons/wi";

const configs = {
    temperature: {
        icon: (s) => <WiThermometer size={s} />,
        label: "Temperature", color: "#e85d04", bg: "#fff3e0", unit: "°C",
    },
    humidity: {
        icon: (s) => <WiHumidity size={s} />,
        label: "Humidity", color: "#00bcd4", bg: "#e0f7fa", unit: "%",
    },
}

```

```

humidity: {
  icon: (s) => <WiHumidity size={s} />,
  label: "Humidity", color: "#0077b6", bg: "#e8f4f8", unit: "%",
},
};

```

```

export default function SensorCard({ type, value }) {
  const c = configs[type];
  const maxVal = type === "temperature" ? 60 : 100;
  const pct = value !== null && value !== undefined ? Math.min((value / maxVal) * 100, 100) : 0;
  return (
    <div className="sensor-card card">
      <div className="sc-top">
        <div className="sc-icon" style={{ color: c.color, background: c.bg }}>{c.icon(28)}</div>
        <span className="live-badge">Live</span>
      </div>
      <h3 className="sc-label">{c.label}</h3>
      <p className="sc-value" style={{ color: c.color }}>
        {value !== null && value !== undefined ? value.toFixed(1) : "--"}{c.unit}
      </p>
      <div className="sc-bar-track">
        <div className="sc-bar-fill" style={{ width: `${pct}%`, background: c.color }} />
      </div>
    </div>
  );
}

```

```

export function IndividualSensorCard({ sensor }) {
  const isNull = (v) => v === "null" || v === null || v === undefined;
  const hasTemp = !isNull(sensor.t);
  const hasHum = !isNull(sensor.h);
  const active = hasTemp || hasHum;
  const tVal = hasTemp ? Number(sensor.t) : null;
  const hVal = hasHum ? Number(sensor.h) : null;
  return (
    <div className={`ind-sensor-card ${active ? "active" : "inactive"}`} >
      <div className="isc-header">
        <span className="isc-label">Sensor {sensor.sensor}</span>
        <span className={`isc-dot ${active ? "on" : "off"}`} />
      </div>
    </div>
  );
}

```

```

    </div>
    <div className="isc-readings">
      <div className="isc-metric">
        <WiThermometer size={20} style={{ color: "#e85d04" }} />
        <span className="isc-val" style={{ color: hasTemp ? "#e85d04" : "#bbb" }}>
          {tVal !== null ? tVal.toFixed(1) : "--"}
        </span>
        <span className="isc-unit">°C</span>
      </div>
      <div className="isc-metric">
        <WiHumidity size={20} style={{ color: "#0077b6" }} />
        <span className="isc-val" style={{ color: hasHum ? "#0077b6" : "#bbb" }}>
          {hVal !== null ? hVal.toFixed(1) : "N/A"}
        </span>
        <span className="isc-unit">{hasHum ? "%" : ""}</span>
      </div>
    </div>
    {!active && <div className="isc-offline">Tidak Aktif</div>}
  </div>
);
}

```

File: `react-dashboard/src/components/SensorChart.jsx`

Deskripsi: Komponen grafik garis menggunakan Recharts untuk visualisasi tren suhu dan kelembaban.

```

import React from "react";
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Legend } from "recharts";
import { IoTrendingUp } from "react-icons/io5";

export default function SensorChart({ data }) {
  const chartData = data.map((entry) => ({
    waktu: entry.waktu || "",
    avg_t: typeof entry.avg_t === "number" ? entry.avg_t : null,
    avg_h: typeof entry.avg_h === "number" ? entry.avg_h : null,
  }));

```

```
}});
```

```
const CustomTooltip = ({ active, payload, label }) => {
  if (!active || !payload?.length) return null;
  return (
    <div className="ct-tooltip">
      <p className="ct-time">{label}</p>
      {payload.map((e, i) => (
        <p key={i} style={{ color: e.color }}>
          {e.name === "avg_t" ? "Suhu" : "Kelembaban"}: <strong>{e.value?.toFixed(1)}{e.name === "avg_t" ? "°C" : "%"}</strong>
        </p>
      ))}
    </div>
  );
};
```

```
return (
  <div className="chart-card card">
    <div className="chart-header">
      <IoTrendingUp size={22} className="chart-icon" />
      <div>
        <h3 className="chart-title">Tren Suhu & Kelembaban</h3>
        <p className="chart-sub">Pemantauan real-time rata-rata sensor</p>
      </div>
    </div>
    {chartData.length === 0 ? (
      <div className="chart-empty">Belum ada data historis...</div>
    ) : (
      <ResponsiveContainer width="100%" height={220}>
        <LineChart data={chartData} margin={{ top: 10, right: 10, left: -20, bottom: 0 }}>
          <CartesianGrid strokeDasharray="3 3" stroke="#eee" />
          <XAxis dataKey="waktu" stroke="#999" fontSize={11} interval="preserveStartEnd" />
          <YAxis stroke="#999" fontSize={11} domain={["auto", "auto"]} />
          <Tooltip content={<CustomTooltip />} />
          <Legend formatter={(value) => value === "avg_t" ? "Suhu (°C)" : "Kelembaban (%")} />
          <Line type="monotone" dataKey="avg_t" stroke="#f97316" strokeWidth={2} dot={{ fill: "#f97316", r: 3 }} connectNulls />
          <Line type="monotone" dataKey="avg_h" stroke="#0077b6" strokeWidth={2} dot={{ fill: "#0077b6", r: 3 }} connectNulls />
        </LineChart>
      </ResponsiveContainer>
    )
  )
);
```

```
    })  
  </div>  
);  
}
```

File: react-dashboard/src/components/BlowerControl.jsx

Deskripsi: Komponen slider PWM untuk mengontrol kecepatan blower.

```
import React, { useState, useEffect } from "react";  
import { ref, set } from "firebase/database";  
import { database } from "../firebase";  
import toast from "react-hot-toast";  
  
export default function BlowerControl({ id, name, speed, onDummyChange, onSpeedChange }) {  
  const [localSpeed, setLocalSpeed] = useState(speed || 0);  
  const MAX_RPM = 6000;  
  
  useEffect(() => { setLocalSpeed(speed || 0); }, [speed]);  
  const rpm = Math.round((localSpeed / 100) * MAX_RPM);  
  
  const handleSliderChange = (e) => { setLocalSpeed(parseInt(e.target.value, 10)); };  
  
  const handleSliderRelease = async (e) => {  
    const finalSpeed = parseInt(e.target.value, 10);  
    if (onSpeedChange) { onSpeedChange(finalSpeed); return; }  
    if (onDummyChange) { onDummyChange(finalSpeed); return; }  
    try {  
      await set(ref(database, `/blowers/${id}/speed`), finalSpeed);  
      toast.success(`${name} speed set to ${finalSpeed}%`);  
    } catch (err) {  
      toast.error("Failed to update blower speed!");  
      setLocalSpeed(speed);  
    }  
  };  
};
```

```

return (
  <div className="blower-control-card">
    <div className="bc-header">
      <span className="bc-subtitle">AIRFLOW MANAGEMENT</span>
      <div className="bc-title-row">
        <div className="bc-title-col"><h2>{name}</h2><h2>Control</h2></div>
        <div className="bc-rpm-col">
          <span className="bc-rpm-value">{rpm.toLocaleString()}</span>
          <span className="bc-rpm-label">RPM</span>
        </div>
      </div>
    </div>
    <div className="bc-slider-container">
      <input type="range" min="0" max="100" value={localSpeed}
        onChange={handleSliderChange} onMouseUp={handleSliderRelease} onTouchEnd={handleSliderRelease}
        className="bc-slider"
        style={{ background: `linear-gradient(to right, #d97706 0%, #d97706 ${localSpeed}%, #e5e7eb ${localSpeed}%, #e5e7eb 100%)` }}
      />
      <div className="bc-slider-labels"><span>OFF</span><span>50%</span><span>MAX</span></div>
    </div>
  </div>
);
}

```

File: react-dashboard/src/components/LoginPage.jsx

Deskripsi: Halaman login dengan Firebase Authentication, validasi error, dan UI modern.

```

import React, { useState } from "react";
import { WiDaySunny } from "react-icons/wi";
import { IoMailOutline, IoLockClosedOutline, IoLogInOutline } from "react-icons/io5";
import { useAuth } from "../contexts/AuthContext";
import toast from "react-hot-toast";

```

```

export default function LoginPage() {
  const { login } = useAuth();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [isLoading, setIsLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!email || !password) { toast.error("Masukkan email dan password!"); return; }
    setIsLoading(true);
    try {
      await login(email, password);
      toast.success("Login berhasil!");
    } catch (err) {
      let msg = "Login gagal!";
      if (err.code === "auth/user-not-found") msg = "Akun tidak ditemukan.";
      else if (err.code === "auth/wrong-password") msg = "Password salah.";
      else if (err.code === "auth/invalid-email") msg = "Format email tidak valid.";
      else if (err.code === "auth/invalid-credential") msg = "Email atau password salah.";
      else if (err.code === "auth/too-many-requests") msg = "Terlalu banyak percobaan.";
      toast.error(msg);
    } finally { setIsLoading(false); }
  };

  return (
    <div className="login-page">
      <div className="login-bg-orbs">
        <div className="login-orb orb-1" />
        <div className="login-orb orb-2" />
        <div className="login-orb orb-3" />
      </div>
      <div className="login-card">
        <div className="login-brand">
          <div className="login-logo-circle"><WiDaySunny size={48} /></div>
          <h1 className="login-title">Solar Dryer IoT</h1>
          <p className="login-subtitle">Masuk ke dashboard monitoring</p>
        </div>
        <form className="login-form" onSubmit={handleSubmit}>
          <div className="login-field">

```

```

        <label className="login-label" htmlFor="login-email">Email</label>
        <div className="login-input-wrap">
            <IoMailOutline className="login-input-icon" />
            <input id="login-email" type="email" className="login-input" placeholder="nama@email.com"
                value={email} onChange={(e) => setEmail(e.target.value)} disabled={isLoading} />
        </div>
    </div>
    <div className="login-field">
        <label className="login-label" htmlFor="login-password">Password</label>
        <div className="login-input-wrap">
            <IoLockClosedOutline className="login-input-icon" />
            <input id="login-password" type="password" className="login-input" placeholder="••••••••"
                value={password} onChange={(e) => setPassword(e.target.value)} disabled={isLoading} />
        </div>
    </div>
    <button type="submit" className={`login-submit ${isLoading ? "loading" : ""}`} disabled={isLoading}>
        {isLoading ? <span className="login-spinner" /> : <><IoLogInOutline size={20} /><span>Masuk</span></>}
    </button>
</form>
<div className="login-footer"><p>Solar Dryer IoT Monitoring System</p></div>
</div>
</div>
);
}

```